Spring Boot CRUD Operations

What Exactly Is a CRUD Application?

Let’s understand CRUD from first principles

Almost every application does four basic things with data:

```
Create data
Read data
Update data
Delete data
```

CRUD is not just beginner content

Many students think CRUD is basic.

CRUD is the foundation of most backend systems.

Example:

```
YouTube video system
Create video
Read video
Update title/description
Delete video
```

```
Instagram post system
Create post
Read post
Update caption
Delete post
```

CRUD is the basic skeleton of backend development.

Student Management API

CRUD Mapping with HTTP Methods

Now connect CRUD with REST API style.

CRUD Operation	HTTP Method	Example URL	Meaning
Create	POST	`/api/students/`	Add new student
Read All	GET	`/api/students/`	Get all students
Read One	GET	`/api/students/{id}`	Get one student
Update	PUT	`/api/students/{id}`	Replace/update student details
Hard Delete	DELETE	`/api/students/{id}`	Remove permanently

What features will we build?

We will build six main APIs.

1. Create Student

Admin can add a new student.

```
POST /api/students/
```

Example request body:

```
{
  "name":"Rahul Sharma",
  "email":"rahul@example.com",
  "course":"Spring Boot",
  "age":22,
  "rollNo" : 101
}
```

This will insert a row into the database.

2. Get All Students

Admin can see all active students.

```
GET /api/students/
```

---

3. Get Student by ID

Admin can fetch one student.

```
GET /api/students/1
```

Here, `1` is the student ID.

This will help you explain:

```
@PathVariable
```

Meaning:

> Value comes from URL

---

4. Update Student

Admin can update student details.

```
PUT /api/students/1
```

Example:

```
{
  "name":"Rahul Sharma",
  "email":"rahul.updated@example.com",
  "course":"Spring Data JPA",
```

```
  "age": 23,
  "rollNo" : 101
}
```

Logic:

```
First find student by ID
Then update fields
Then save updated student
```

This is a very important backend pattern.

---

5. Delete Student

Admin can permanently delete a student.

```
DELETE /api/students/1
```

This will actually remove the row from the database.

So we will show both delete styles:

```
Soft delete → row remains, isDeleted becomes true
Hard delete → row is permanently removed
```

This makes the project more real-world.

What will our database table look like?

The table name can be:

```
students
```

Columns:

Column	Type	Meaning
id	BIGINT	Unique student ID
name	VARCHAR	Student name
email	VARCHAR	Student email
course	VARCHAR	Student course
age	INT	Student age
roll_no	INT	Rolll Number

In Java, this table will be represented by a class:

```
public class Student {
    private String name;
    private String email;
    private int rollNo;
    private int age;
    private String subject;
    }
```

This is where students will see:

> Java object database table se connect ho sakta hai.

That is the foundation of JPA.

Why do we need architecture?

Imagine we write the full CRUD code inside one class:

```
@RestController
public class StudentController {

    // receive request
    // validate data
    // write business logic
```

```
    // talk to database
    // update student
    // delete student
}
```

Technically, for a very small demo, this may work.

But this is not a good backend design.

Why?

Because one class is doing everything:

```
Receiving HTTP request
Processing business logic
Talking to database
Deciding response
Managing update/delete rules
```

This becomes messy very quickly.

So in real applications, we divide responsibility.

That is the main reason we create layers.

The final architecture

Our application will follow this flow:

```
Postman
   ↓
Controller
   ↓
Service
   ↓
Repository
   ↓
Database
```

This is the core structure of the project.

Step 1: Postman sends request

Postman is acting like a client.

In real life, this request may come from:

```
React frontend
Angular frontend
Mobile app
Another backend service
Postman
```

Postman sends data to our Spring Boot application.

Step 2: Controller receives request

The first Java layer that handles this request is:

```
Controller
```

Controller's job is not to write all logic.

Controller's job is mainly:

```
Receive request
Read input data
Call service layer
Return response
```

So the controller is like the entry gate of the application.

Step 3: Service handles business logic

Controller will call the service:

```
studentService.createStudent(student);
```

The service layer contains the actual business rules.

For example:

```
Should this student be saved?
Should deleted students be hidden?
How should update happen?
What should happen if student ID does not exist?
Soft delete or hard delete?
```

In our first CRUD app, business logic will be simple.

Step 4: Repository talks to database

The service layer will call repository:

```
studentRepository.save(student);
```

Repository is responsible for database operations.

Important thing:

We will not write SQL manually for basic operations.

Spring Data JPA gives us ready-made methods like:

```
save()
findAll()
findById()
deleteById()
```

So repository is the layer that connects Java code with database operations.

Step 5: Database stores the data

Finally, the data reaches MySQL.

A new row is inserted into the `students` table.

Then response travels back:

```
Database
   ↓
Repository
   ↓
Service
   ↓
Controller
   ↓
Postman
```

Layer-by-layer responsibility

This table should definitely be shown in your video:

Layer	Main Responsibility	Example
Postman	Sends HTTP request	`POST /api/students`
Controller	Handles API request	`StudentController`
Service	Contains business logic	`StudentService`
Repository	Performs database operations	`StudentRepository`
Database	Stores actual data	MySQL `students` table
Entity/Model	Represents table as Java object	`Student` class

Where does Entity/Model fit?

In the diagram, we usually write:

```
Controller → Service → Repository → Database
```

But there is one more important part:

```
Entity / Model
```

The entity is not exactly a “flow layer” like controller or service.

Instead, it is the object that travels between layers.

Example:

```
Student student
```

This `Student` object contains data like:

```
name
email
course
age
```

And JPA maps this object to the database table.

```
Controller receives Student data.
Service works with Student object.
Repository saves Student object.
JPA converts Student object into students table row
```

Till now, we learned DI using console examples. Today, you will see the same DI in a real Spring Boot backend application.

Setting up the Project :

Dependencies We Need

For this CRUD project, add these dependencies:

```
Spring Web
Spring Data JPA
MySQL Driver
```

Why Spring Web?

Spring Web is needed because we want to build APIs

Without Spring Web, our application will be like a normal backend app, but it will not expose HTTP endpoints like:

```
GET /api/students
POST /api/students
PUT /api/students/1
DELETE /api/students/1
```

Spring Web gives us annotations like:

```
@RestController
@RequestMapping
@GetMapping
@PostMapping
@PutMapping
@DeleteMapping
@PatchMapping
```

Why Spring Data JPA?

Spring Data JPA is needed because we want to perform database operations without writing basic SQL manually.

So Spring Data JPA reduces boilerplate database code.

Why MySQL Driver?

Our Spring Boot app is Java code.

MySQL is a separate database.

So Java needs a driver to communicate with MySQL.

That is why we add:

```
MySQL Driver
```

RUN THE EMPTY APP WITH HELLO WORLD :

```
The app failed because you added Spring Data JPA + MySQL Driver,
so Spring Boot assumed:

“This project wants to connect to a database.”

Then during startup, Spring Boot tried to auto-create a DataSource for MySQL.
But your project does not yet have database details like URL, username,
and password, so it failed with:

Failed to configure a DataSource: 'url' attribute is not specified

Reason: Failed to determine a suitable driver class
```

Spring Boot does this automatically:

```
Spring Web present       → start embedded Tomcat on port 8080
Spring Data JPA present  → configure JPA, Hibernate, EntityManager
MySQL Driver present     → configure MySQL DataSource
```

But it cannot connect because you have not provided:

```
spring.datasource.url
spring.datasource.username
spring.datasource.password
```

So the app crashes before fully starting.

Spring Boot auto-configuration works based on the JAR dependencies present in your project. For example, if a database library is present and you have not manually configured a database bean, Spring Boot tries to configure one automatically

So in your project:

```
spring-boot-starter-web
```

tells Spring Boot:

```
This looks like a web application.

I should configure Tomcat, DispatcherServlet, web MVC setup,
JSON support, etc.
```

And:

```
spring-boot-starter-data-jpa
mysql-connector-j
```

tells Spring Boot:

```
This looks like a database/JPA application.
I should configure DataSource, Hibernate, EntityManagerFactory, TransactionManager, Repos
itory support, etc.
```

It showed failure while creating `entityManagerFactory` because the `DataSource` could not be created without DB URL details.

Why Do We Need Folder Structure?

Imagine we write everything inside the main package directly:

```
com.coderarmy.studentcrud
│
├── StudentcrudApplication
├── Student
├── StudentController
├── StudentService
├── StudentRepository
```

This will still work.

But as soon as the project grows, it becomes messy.

Suppose later we add:

```
Teacher
Course
Batch
Payment
Attendance
Enrollment
```

Then the package becomes crowded.

So instead of keeping all classes together, we divide them based on responsibility.

That is why we create packages like:

```
controller
service
repository
entity
```

Each package has a clear purpose.

Main Application Class

This class is the starting point of the Spring Boot application.

It should stay in the root package:

```
com.coderarmy.studentcrud
```

Why?

Because `@SpringBootApplication` includes component scanning.

By default, Spring scans the package where this class is present and its sub-packages.

Entity Package

Package:

```
com.coderarmy.studentcrud.entity
```

Class:

```java
Student.java

@Entity
public class Student {
    private String name;
    private String email;
    private int rollNo;
    private int age;
    private String subject;
    private boolean isDeleted;
}
```

The entity package contains classes that represent database tables.

Why Are We Using MySQL?

For this project, we will use MySQL because:

```
It is popular.
It is beginner-friendly.
```

```
It is commonly used with Spring Boot.
It is easy to inspect data using MySQL Workbench or terminal.
```

What Exactly Are We Creating?

We are creating a database named:

```
student_crud_db
```

Inside this database, later JPA will create a table named:

```
students
```

So the structure will become:

```
MySQL Server
   ↓
student_crud_db
   ↓
students table
   ↓
student rows
```

Important point:

At this stage, we will only create the database

We will not manually create the table.

The table can be created automatically by JPA when we run the application with proper configuration.

Setting up MySQL :

- MySQL Server = actual database engine
- MySQL GUI Application = GUI application to manage that database

GUI application for MySQL:

- Work Bench (Official MySQL GUI)
- Sequel Ace (For mac)
- DBeaver (Light cross platform)

Install both on your mac/PC by following instructions.
If i teach the install instructions here, this will be unnecesssary.
I will include this in notes or follow chatGPT / google instructions.

Installing MySQL Server, MySQL Client, and MySQL Workbench

1. First understand the difference

Before using MySQL with a Spring Boot CRUD application, we need to understand three things:

```
MySQL Server  = actual database engine
MySQL Client  = terminal command used to connect to MySQL Ser
ver
MySQL Workbench = GUI application used to manage MySQL visual
ly
```

Part A: Install MySQL on macOS

Install MySQL using Homebrew (Install homebrew if not already)

Open Terminal and run:

```
brew install mysql
```

This installs:

```
MySQL Server
MySQL command-line client
```

Now start MySQL Server:

```
brew services start mysql
```

Check whether MySQL is running:

```
brew services list
```

You should see MySQL with status:

```
started
```

You can also check the installed MySQL version:

```
mysql --version
```

Set root password on macOS

Run:

```
mysql_secure_installation
```

It may ask for the current root password.

If you have not set any password yet, just press:

```
Enter
```

Then follow the prompts.

Recommended beginner-friendly answers:

```
Set root password? → Yes
New password → your password
Remove anonymous users? → Yes
Disallow root login remotely? → Yes
Remove test database? → Yes
Reload privilege tables? → Yes
```

Now test login:

```
mysql -u root -p
```

Enter your password.

If login is successful, you will see:

```
mysql>
```

That means your MySQL Server is working and your terminal client is connected.

Useful macOS MySQL commands

```
Start MySQL Server: brew services start mysql
Stop MySQL Server: brew services stop mysql
Restart MySQL Server: brew services restart mysql
Check service status: brew services list
Login : mysql -u root -p
Exit : exit;
```

Part B: Install MySQL on Windows

Step 1: Download MySQL Installer

Download **MySQL Installer for Windows** from the official MySQL website.

Choose:

```
MySQL Installer
```

During installation, select:

```
Developer Default
```

This usually installs:

```
MySQL Server
MySQL Workbench
MySQL Shell
MySQL command-line client
```

Step 2: Configure MySQL Server on Windows

During setup, keep these settings:

```
Config Type: Development Computer
Port: 3306
Authentication Method: Strong Password Encryption
Username: root
Password: your password
Run MySQL as Windows Service: Yes
Start MySQL Server at System Startup: Yes
```

The installer requires assigning a root password during server configuration.

Step 3: Start or stop MySQL Server on Windows

Open **Command Prompt** or **PowerShell** as Administrator.

Check MySQL service name:

```
Get-Service *mysql*
```

You may see a service name like:

```
MySQL80
```

Start MySQL Server:

```
net start MySQL80
```

Stop MySQL Server:

```
net stop MySQL80
```

If your service name is different, replace `MySQL80` with the name shown on your system.

Example:

```
net start MySQL
```

or:

```
net start MySQL84
```

MySQL's Windows documentation explains that MySQL Server can run as a Windows Service and can be started from the Services utility or command line.

Step 4: Open MySQL from Windows terminal

Open Command Prompt or PowerShell.

Run:

```
mysql -u root -p
```

Enter the root password you created during installation.

If login is successful, you will see:

```
mysql>
```

The default MySQL port is `3306`, and Workbench's Standard TCP/IP connection uses host, port, username, and password to connect to MySQL Server.

```
mysql --version
```
